

Safe Edges

2025

Smart Contract Audit REPORT

ASSESSED ON Nov, 2025

Luxxcoin

Security Assessment



SAFEEDGES.IN

LuxxCoin Backend Draft Audit Report

Table of Contents

1. Executive Summary

2. Scope

3. Methodology

4. Severity Classification

5. Findings Overview

C-01 .Authentication Bypass via User-Controlled Public Key in Ed25519Auth

6. Detailed Findings (with code, impact, and recommendations)

7. Conclusion

About Safe Edges

Safe Edges was established in 2023 by industry-renowned blockchain security expert. We are a trusted audit partner for leading Web3 protocols, decentralized finance (DeFi) platforms, NFT marketplaces, and Web2 enterprises.

Our mission is to secure the future of blockchain technology by providing comprehensive security solutions that protect digital assets and ensure the integrity of blockchain ecosystems.

With a team of top-ranked security researchers and former security engineers from major enterprises, we bring unparalleled expertise to every project we undertake.

Executive Summary

Item	Details
Scope	Review of the following files: https://github.com/Luxxcoin-team/Luxx-Backend/tree/main/src
Timeframe	November 2025
Methodology	Comprehensive manual code review, static analysis, threat-modeling, and best-practice compliance verification
Severity Summary	MEDIUM
Overall Risk	MEDIUM
Status	Under Review

Luxxcoin (\$LUX) is a Solana-based MemeFi project that turns meme culture into a rules-based, on-chain competition.

Luxxcoin (\$LUX) is a meme-powered Solana token that transforms HODLing into a recurring, live leaderboard game. Every two months, the top 50 wallets are rewarded with a massive share of 1.0 Billion \$LUX. sbased on how much you hold, how long you've held, and how loyal you've been. There are no dev bags. No taxes. No VC allocation. Just pure community. Pure rewards. Pure meme competition

Techniques and Methods

- Code quality checks
- Best practices & documentation review
- Gas efficiency analysis
- Reentrancy & ERC compliance checks

The Methodology

Luxx engaged SafeEdges to perform a security audit of the Luxx backend code.

The following sections describe how the engagement was structured and executed.

2.1 Preparation Phase

The SafeEdges team thoroughly reviewed all provided materials and analyzed the backend source code to gain a complete understanding of its architecture, workflows, and functionalities.

They examined all major components including service modules, controllers, APIs, database interactions, and third-party integrations to systematically identify potential security vulnerabilities.

The analysis prioritized issues with significant impact on data integrity, user privacy, authentication, authorization, and overall system security. To validate findings, the team conducted hands-on testing in a controlled environment, performing rigorous functional and security validations throughout the audit.

A testing window was mutually agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was finalized:

IN-SCOPE ASSETS

<https://github.com/arcana-network/ca-wallet>

2.1.2 Documentation

Documentation was not required, as the backend implementation and source code were sufficient for understanding the project.

2.1.3 Audit Goals

SafeEdges uses a combination of proprietary automated tools and advanced manual techniques to conduct a comprehensive security assessment of backend systems.

Most of the audit focuses on manual review of the backend code, following OWASP ASVS guidelines along with an extended, internally developed checklist tailored for backend security.

The team emphasizes understanding the systems core logic, preparing detailed test scenarios, and analyzing workflows, API flows, and business logic to uncover potential vulnerabilities.

2.2 Retesting Phase

Luxx is actively collaborating with SafeEdges to validate the remediations implemented for the identified vulnerabilities.

2.3 Vulnerability Classification and Severity

SafeEdges follows OWASPs Risk Rating Methodology to determine the severity of discovered vulnerabilities.

This approach considers two main factors Likelihood and Impact each evaluated as Low, Medium, or High. The evaluation includes threat agent characteristics, vulnerability factors, and both technical and business impacts.

The final severity rating is derived by combining the likelihood and impact assessments.

Analysis Types

Structural Analysis – Reviewed design patterns & contract structure

Static Analysis – Automated scanning for vulnerabilities

Manual Analysis – Line-by-line code review

Gas Consumption – Optimization analysis

Tools: Remix IDE, Foundry, Slither, Mythril, Solhint

Severity Levels

Critical

Issues that can completely compromise protocol integrity or allow direct theft/loss of funds. Must be fixed before deployment.

High

Exploitable vulnerabilities affecting critical functionality, potentially causing loss of assets, broken pools, or bypassed core logic. Must be addressed immediately.

Medium

Weaknesses causing errors, inconsistent behavior, or revenue loss under certain conditions. Should be fixed but not catastrophic.

Low

Minor issues with limited impact, such as edge-case bugs or inefficient gas usage. Worth fixing for reliability.

Informational

Cosmetic or documentation issues. Do not pose direct threats but may confuse users or developers.

Issue Status

Open	Resolved

Security vulnerabilities identified that must be resolved and are currently unresolved.	Security vulnerabilities that have been identified and successfully fixed or mitigated.
Acknowledged	Partially Resolved
Vulnerabilities which have been acknowledged but are yet to be resolved.	Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

ID	Title	Severity
C-01	Authentication Bypass via User-Controlled Public Key in Ed25519Auth	CRITICAL

[CRITICAL] Authentication Bypass via User-Controlled Public Key in Ed25519Auth

Severity: Critical

Impact: Access control compromise, admin endpoints fully exposed

Affected component: Ed25519Auth middleware

Category: Authentication bypass / Broken cryptographic design

OWASP: A07 – Identification & Authentication Failures

Summary

The ed25519Auth middleware accepts the following headers:

X-Gov-Public-Key

X-Gov-Signature

X-Gov-Challenge

X-Gov-Timestamp

It verifies that the provided signature is valid for the message `challenge:timestamp` **using the public key supplied by the caller**. The configured trust anchor (`config.auth.govPublicKey`) is never used. As a result, an attacker can:

1. Generate any Ed25519 key pair.
2. Choose an arbitrary `challenge` string and a current `timestamp`.
3. Sign `challenge:timestamp` with their private key.
4. Call any admin route, providing the attacker's public key in `X-Gov-Public-Key`.

Verification succeeds because the signature is valid for the attacker-supplied key — even though that key is not trusted by the server. This yields a **complete authentication bypass**.

Example (excerpt of vulnerable logic):

```
ed25519Auth(req, res, next) {
  const signature = req.headers['x-gov-signature'];
  const publicKey = req.headers['x-gov-public-key'];
  const challenge = req.headers['x-gov-challenge'];
  const timestamp = req.headers['x-gov-timestamp'];
  // ...
  const message = `${challenge}:${timestamp}`;
  const isValid = this.verifyEd25519Signature(message, signature, publicKey);
  // ...
}
```

Impact

All admin-only endpoints (e.g. `POST /rounds/:id/process`, scoring/config endpoints, scheduler triggers, payouts) become accessible to anyone who can send forgable headers. Attackers can finalize rounds, change scoring weights, trigger payouts, or perform other privileged actions without authorization. This represents a **full loss of access control** for governance/admin functionality.

Root cause

The middleware verifies signatures using **client-supplied** public keys instead of the **server-configured trusted authority key**. This removes the trust anchor that makes digital signatures meaningful: signatures are only authentic when checked against a public key the server already trusts.

Recommended fixes

1. Pin the trusted public key never accept the public key from the client. Always verify signatures against if

```
(publicKey !== this.govPublicKey) {
```

```
  return res.status(403).json({
```

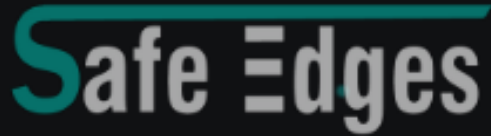
```
    success: false,
```

```
    error: 'Invalid GOV public key',
```

```
    timestamp: new Date().toISOString(),
```

```
  });
```

```
}
```



REVOLUTIONIZING BLOCKCHAIN AUDITS

Founded in 2023, Safe Edges is the largest blockchain security company that ensures the safety and reliability of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative technology, we support the success of our clients with best-in-class security, all whilst realizing our overarching vision: ensuring blockchain remains secure and trustworthy.

THANK YOU FOR WORKING WITH US



SAFEEDGES.IN